&¡ ChargeShare

Peer-to-Peer EV Charging Network

Project Documentation • Tech Stack • Features • Commands

Project Type

Full-Stack Web Application

Stack

React + Spring Boot + MySQL

Generated: 19 July 2026

Tech Stack

Full technology breakdown of the ChargeShare platform

Ø=Y¥þ FRONTEND

React.js (via Vite)

- **Framework:** React 18 — all screens built as JSX components with hooks
- **Routing:** Hash-based routing (#/explore, #/confirmed, etc.) — no react-router needed
- **State Management:** React Context API (useApp hook) — shared global state across all components

Vite

- **Role:** Lightning-fast dev server with Hot Module Replacement (HMR)
- **Build Output:** Optimized production bundle in /dist — 257 kB JS, 7.9 kB CSS
- **Proxy:** Proxies /api/* calls to Spring Boot on port 8080 — zero CORS issues

Tailwind CSS (CDN)

- **Usage:** Utility-first styling — dark mode, spacing, layout, responsiveness
- **Config:** Custom design tokens: Electric Green (#00ff87), Deep Navy (#051424)

Vanilla CSS (src/style.css)

- **Glassmorphism:** Frosted glass cards with backdrop-filter blur
- **Animations:** Pulse glow, shimmer scan, breathing glow, marker pulse keyframes
- **Leaflet Theming:** Custom dark-mode map popups, zoom controls, marker styles

Leaflet.js

- **Purpose:** Interactive real-time map showing charger locations as colored markers
- **Markers:** Available (green glow), Busy (muted), GPS Location (red pulse)

Google Fonts & Material Symbols

- **Fonts:** Geist (headings), Inter (body), JetBrains Mono (labels/codes)
- **Icons:** Material Symbols Outlined — used throughout all UI components

&™þ BACKEND

Spring Boot 4 (Java 21)

- **Role:** REST API server — handles all database operations
- **Endpoints:** /api/db (GET), /api/sync (POST), /api/reset (POST)
- **Port:** 8080 (proxied from Vite frontend on 5173)
- **CORS:** Resolved via Vite proxy — no separate CORS config needed

Spring Data JPA + Hibernate 7

- **ORM:** Maps Java entity classes to MySQL tables automatically
- **DDL Auto:** hibernate.ddl-auto = update — auto-migrates schema on startup
- **Null Safety:** All numeric fields use boxed types (Double, Integer) to allow NULLs

Gradle 9

- **Role:** Build tool — compiles Java, manages dependencies, runs bootRun
- **Key Tasks:** clean, compileJava, bootRun, build

Ø=YAþ DATABASE

MySQL 8.0

- **Host:** 127.0.0.1:3306
- **Schema:** chargeshare (auto-created on first startup)
- **Tables:** users, chargers, reviews, driver_history, owner_bookings, active_session, upcoming_booking

localStorage (Browser)

- **Role:** Client-side fallback — used when backend is unreachable
- **Key:** chargeshare_db — full JSON snapshot of app state
- **Sync:** Always written alongside every /api/sync call

Features

Complete feature list of the ChargeShare platform

Ø=Üd USER & AUTHENTICATION

- **Dynamic Profile:** User can set name, phone number, and vehicle details via Profile screen
- **Dual Role System:** Single app supports both EV Driver and Charger Host roles
- **Role Switching:** Seamlessly switch between Driver and Host dashboards from sidebar
- **Name Persistence:** Profile name carries across all screens — no hardcoded defaults
- **Wallet Balance:** Displayed in profile — deducted after charging sessions
- **Reset Options:** Clear saved data or full database reset from Profile screen

Ø=ÍÎ EXPLORE & MAP

- **Live Map:** Leaflet.js interactive map with real-time charger pin markers
- **Charger Cards:** Filterable list with distance, price, rating, status badges
- **GPS Location:** Auto-detects and centers map on user's current location
- **Status Indicators:** Green (Available), Grey (Busy) pulsing map markers
- **Search & Filter:** Filter chargers by connector type, price range, availability

Ø=ÜÊ BOOKING SYSTEM

- **Dynamic Time Picker:** Custom clock input — book any time slot, not just presets
- **Conflict Detection:** Blocks slots already booked or in the past
- **Booking Flow:** Driver books !' Pending !' Host Accepts/Declines !' Driver notified
- **QR Code Display:** Unique QR matrix shown on booking confirmation screen
- **Stall Assignment:** Auto-assigns random stall (e.g. B-04) on booking creation
- **Battery Targets:** Select start % and target % for precise charging

Ø=Ý APPROVAL WORKFLOW

- **Host Requests Panel:** All pending bookings shown in Host Dashboard with Accept/Decline buttons
- **Driver Notification:** Active Booking Reservation banner appears on Driver Dashboard
- **Status States:** Pending !' Accepted (driver) / Confirmed (host) !' Active !' Completed
- **Declined Handling:** Driver sees Declined badge with option to clear and rebook

&¡ CHARGING SESSION

- **Count-Up Timer:** Live stopwatch from 0:00 — tracks actual time elapsed
- **Metered Pricing:** Cost = (elapsed seconds / 3600) × charger kW × price per kWh
- **Battery Progress:** Real-time battery % bar on both Driver and Host dashboards
- **Stop Charging:** Driver or Host can end session — cost written to history
- **Energy Tracking:** kWh delivered calculated and saved to charging history

Ø<Ü HOST DASHBOARD

- **Listings Management:** Add, edit, delete charger listings with live status toggle
- **Earnings Overview:** Total net earnings, energy delivered, host rating displayed
- **Session Monitor:** Live view of active charging session with battery progress bar
- **Booking History:** Completed and declined bookings shown in Past Bookings section
- **Charger Creation:** Full in-place add for price, power, connector, hours, name

Ø=Ð DRIVER DASHBOARD

- **Charging Status:** Live session card or "no session" state with explore CTA
- **Session History:** All past charging sessions with charger name, cost, energy, duration
- **Aggregate Stats:** Total sessions, total kWh consumed, total amount spent
- **Reservation Banner:** Shows active booking with status (Pending / Approved)

' þ REVIEWS

- **Submit Review:** Drivers can rate and review chargers after sessions
- **Tag System:** Quick tags: Fast Charging, Great Location, Clean, etc.
- **Host Rating:** Aggregated from all reviews — shown on host dashboard
- **Review Cards:** Avatar, date, star rating, helpful count displayed per review

Commands & Setup

All commands needed to run, build, and manage the ChargeShare platform

Ø=Þ€ STARTING THE APP

Run Everything (Recommended)

Starts both React frontend (port 5173) and Spring Boot backend (port 8080) concurrently:

```
npm run dev
```

Run Frontend Only

```
npm run frontend
```

Run Backend Only

```
npm run backend
```

(equivalent to: cd backend && gradlew.bat bootRun)

Ø=Y(BUILD COMMANDS

Production Build (Frontend)

Compiles and bundles React app into /dist folder:

```
npm run build
```

Backend Clean + Compile

```
.\backend\gradlew.bat clean compileJava
```

Backend Full JAR Build

```
.\backend\gradlew.bat build
```

Ø=YÀþ DATABASE COMMANDS

Reset Database via API

Clears all tables and re-seeds default user (also available in Profile screen):

```
Invoke-RestMethod -Uri "http://localhost:8080/api/reset" -Method POST
```

Check Database State

```
Invoke-RestMethod -Uri "http://localhost:8080/api/db"
```

MySQL — Connect to Database

```
mysql -u root -pannanya2205 chargeshare
```

MySQL — View All Tables

```
SHOW TABLES;
```

MySQL — View Users

```
SELECT id, name, role, balance FROM users;
```

Ø=Uæ DEPENDENCY COMMANDS

Install Node Dependencies

```
npm install
```

Check Outdated Packages

```
npm outdated
```

View Installed Packages

```
npm list --depth=0
```

Ø<ß URLS

Frontend (React App)

```
http://localhost:5173
```

Backend API Base

```
http://localhost:8080/api
```

DB Endpoint

```
http://localhost:8080/api/db
```

Sync Endpoint

```
http://localhost:8080/api/sync  [POST]
```

Reset Endpoint

```
http://localhost:8080/api/reset  [POST]
```